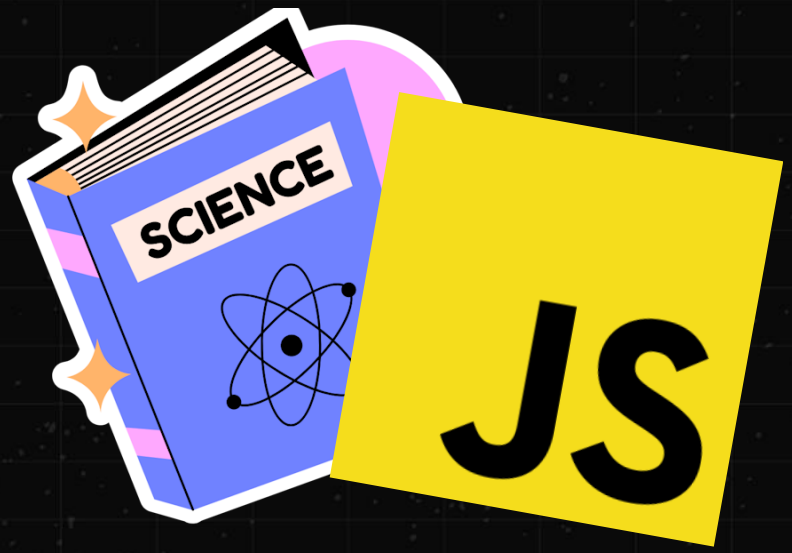
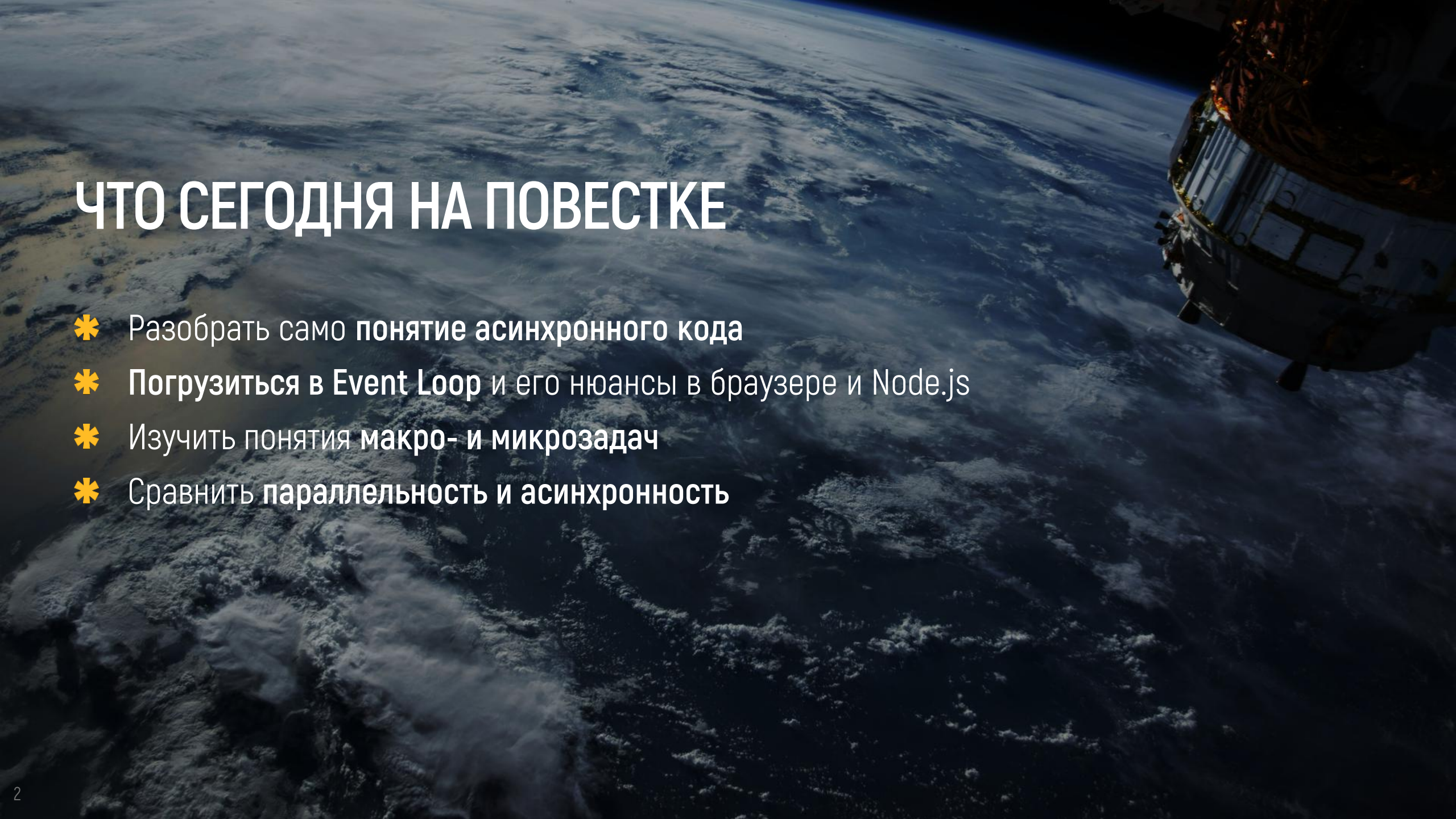
KOBEZZA.LAB

ЛЕКЦИЯ #27

CS BO FRONTEND





ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- Разобрать само **понятие асинхронного кода**
- **Погрузиться в Event Loop** и его нюансы в браузере и Node.js
- Изучить понятия **макро- и микрозадач**
- Сравнить **параллельность и асинхронность**

В ЧЕМ ЗАКЛЮЧАЕТСЯ АСИНХРОННОСТЬ?



ОБЫЧНЫЙ ПОРЯДОК ИСПОЛНЕНИЯ

- Инструкции **выполняются последовательно друг за другом**
- Просто прочитав код, можно с уверенностью сказать,
  что именно произойдёт и в каком порядке
- **Это и есть синхронный код**, его ещё называют нормальным потоком исполнения
- Но всегда ли такое возможно?

ЕСЛИ КОД НУЖНО ВЫПОЛНИТЬ ПО «СОБЫТИЮ»

- Пользователь нажал на кнопку
- Мы отправили запрос на сервер и он прислал ответ
- Нужно выполнить код, только если процессор бездействует
- И так далее



У НАС ЕСТЬ СОБЫТИЯ, СРАБАТЫВАНИЕ КОТОРЫХ СЛОЖНО ПРЕДСКАЗАТЬ...

В ТОЧКУ

- Более того, мы **не можем продолжить выполнять код** в таком случае – ведь он завязан на завершение конкретного события
- Мы должны **заблокировать весь поток исполнения** до получения нужного события
- Заблокированный поток **не делает никакую другую работу**, а просто ждёт

```
function getUserData(id) {
    const request = new XMLHttpRequest();

    // Последний аргумент говорит, что мы должны заблокировать поток
    // после send до получения ответа сервера
    request.open("GET", `/user/${id}`, false);
    request.send(null);

    if (request.status === 200) {
        return request.response;
    }
}

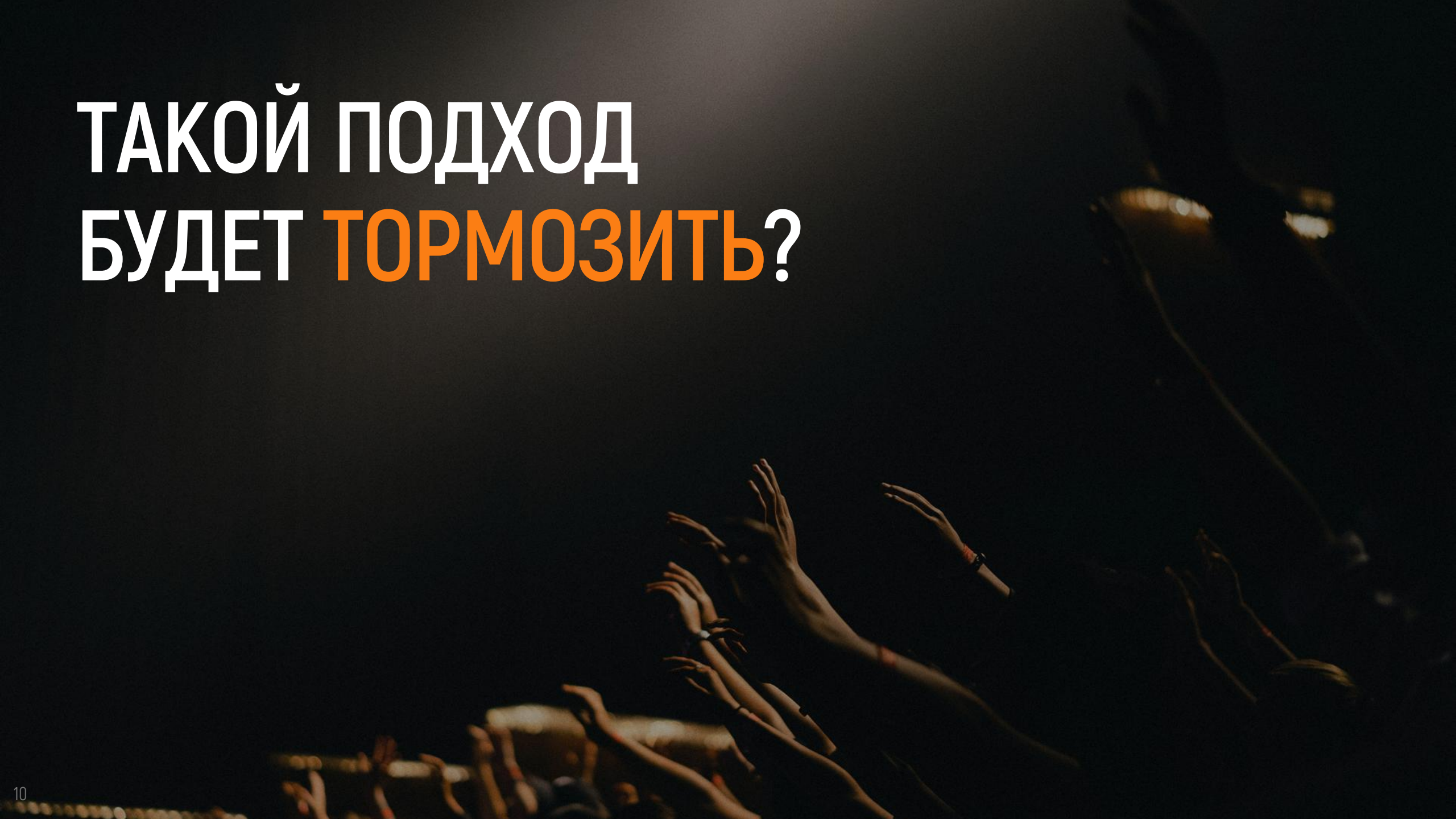
console.log(1);
console.log(getUserData("10"));

// Появится после ответа сервера
console.log(2);
```

РЕФЛЕКСИРУЕМ

- Мы отправили сетевой запрос на удалённый сервер
- И попросили браузер заблокировать наш поток до ответа сервера
- Пока поток заблокирован, никакой другой код в нём не будет исполняться
- Если сервер отвечает долго, то пользователь думает, что сайт «завис»

ТАКОЙ ПОДХОД БУДЕТ ТОРМОЗИТЬ?



СМОТРЯ, ЧТО МЫ ИМЕЕМ В ВИДУ

- **Для пользователя будет ужасный опыт** – кнопки не нажимаются, анимации не проигрываются, никакой интерактивности
- Процессор, напротив, **будет радостно прохлаждаться в бездействии**
- С точки зрения общей производительности, мы здесь больше **зависим от сети и скорости ответа сервера**

КОГДА ВАЖНА ИНТЕРАКТИВНОСТЬ

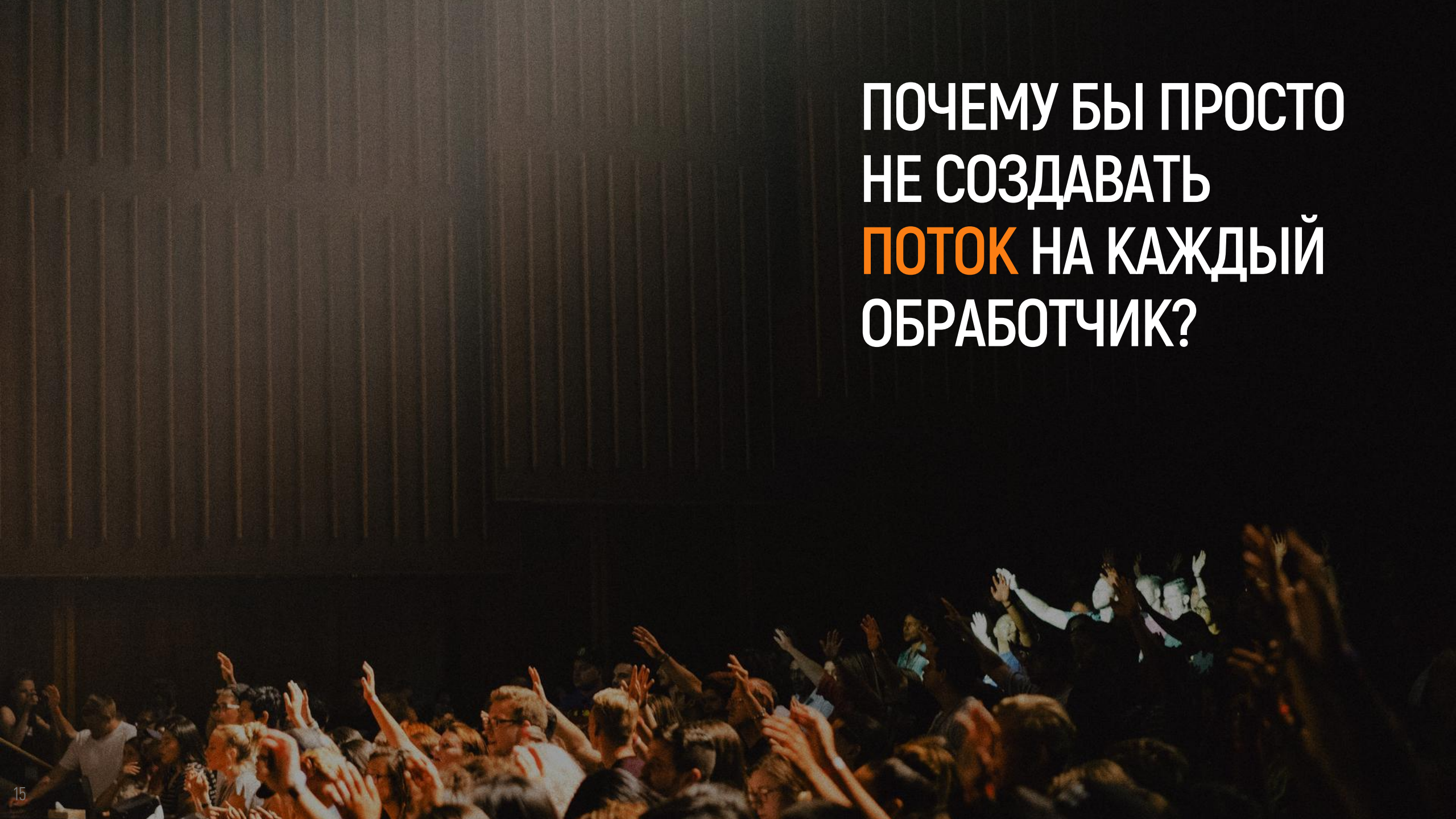
- Для пользователя зачастую **куда важнее интерактивность**,
  а не скорость работы конкретной операции (это не всегда так)
- Важно, чтобы интерфейс не тормозил и моментально
  реагировал на все действия пользователя
- Веб-сервер должен быстро принимать новые запросы и отвечать так,
  чтобы **не создавать очередей и не заставлять пользователя ждать**
- Серверу **важна масштабируемость и эффективное использование ресурсов** –
  особенно когда одновременно работает много пользователей


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ЗАДАЙТЕ СЕБЕ ВОПРОС

- Какая программа для вас будет «быстрее»
- Которая повесит ваш компьютер, но **выполнит работу быстрее**
- Или та, которая покажет красивый прогресс-бар
  и **позволит вам спокойно работать** за компьютером, пока она выполняется
- А теперь ответьте на этот вопрос, если ваш компьютер
  обслуживает одну конкретную задачу без интерактивности

ПОЧЕМУ БЫ ПРОСТО НЕ СОЗДАВАТЬ ПОТОК НА КАЖДЫЙ ОБРАБОТЧИК?



ТАК МОЖНО

- Многие старые веб-серверы использовали такой подход
- Это хороший вариант, **когда обработчиков не очень много**
- Код остаётся **синхронным и простым в отладке**
- А когда обработчиков много, то мы упираемся в **неэффективность такого использования потоков**

В ЧЕМ ПРОБЛЕМА С ПОТОКАМИ

- Потоки созданы для сценариев, когда у нас есть **несколько параллельных ветвей вычислений** (нагрузка на CPU)
- А в сценарии с интенсивным вводом-выводом (I/O) **потоки большую часть времени просто ждут**
- Потоки **реализуются на уровне ОС** и требуют собственный сегмент стека
- Обработчиков много – **закончится память и/или производительность** резко упадёт из-за переключения контекста

ЗЕЛЕНЫЕ ПОТОКИ

- Говоря о потоках, нельзя не упомянуть подход, когда их реализация происходит **на уровне среды исполнения или библиотеки**, а не только на уровне ОС
- Например, **в рамках одного потока ОС среда исполнения создаёт множество легковесных потоков** (green threads) и самостоятельно планирует их выполнение
- Такой подход менее ресурсозатратен и позволяет масштабироваться до **десятков и даже сотен тысяч одновременных потоков**
- Применяется в Go, Erlang и многих библиотеках



А МОЖНО ЛИ ВЫДЕЛИТЬ КОД, КОТОРЫЙ НУЖНО ЗАПУСТИТЬ ПО СОБЫТИЮ В ФУНКЦИЮ?

МОЖНО

- Эту функцию **нужно зарегистрировать как обработчик** некоторого события
- Такие функции называются «колбэками», «хэндлерами» или **функциями обратного вызова**
- После регистрации хэндлера управление возвращается в поток и **он продолжает работу без каких-либо блокировок**
- Как только зарегистрированное событие произойдет, наш **колбэк встанет в очередь на выполнение**

```
function getUserData(id, callback) {
    const request = new XMLHttpRequest();

    // Обратите внимание на последний аргумент
    request.open("GET", `/user/${id}`, true);

    request.onload = function() {
        if (request.readyState === 4 && request.status === 200) {
            callback(request.response);
        }
    };

    request.send(null);
}

console.log(1);
getUserData("10", console.log);
// Появится после 1, а ответ getUserData появится позже
console.log(2);
```

ПОЛУЧАЕТСЯ МЫ ПЕРЕСТАЛИ БЛОКИРОВАТЬ ПОТОК?

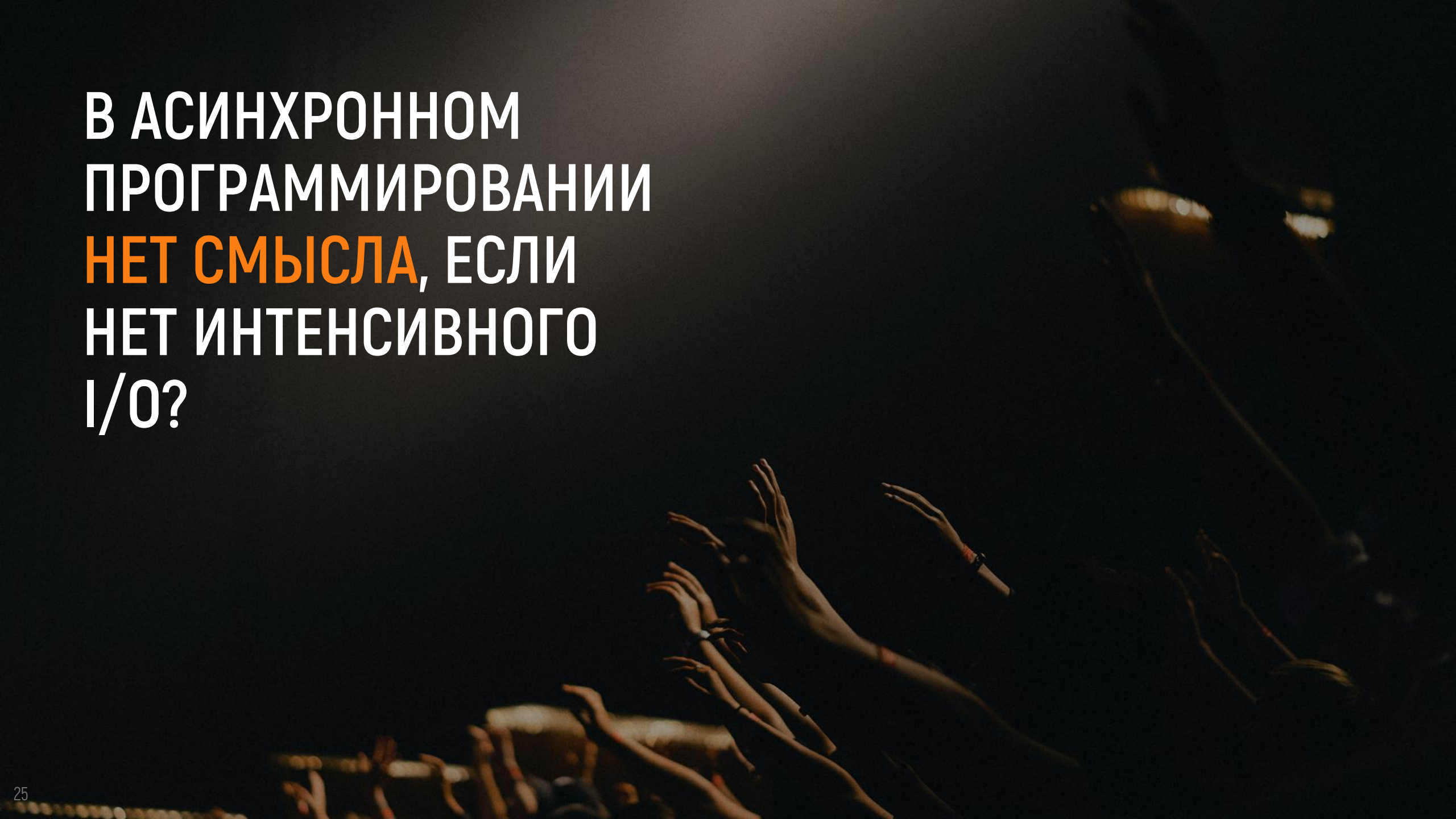


ИМЕННО

- В этом и кроется цель и смысл асинхронного программирования – **не блокировать поток, пока он ожидает некоторого события**
- Отказ от синхронной блокировки потоков ОС для ожидания операций ввода-вывода приводит к **повышению производительности и снижению потребления памяти**
- Но у этого есть и цена: **наш код стал сложнее** – как для написания, так и для чтения и отладки

ПРОБЛЕМА СЛОЖНОСТИ

- Асинхронное программирование очень **усложняет разработку и отладку**
- Любые сценарии **построения асинхронных композиций и обработка ошибок** – боль
- **Для решения этой проблемы** в стандарт JS добавили Promise
- Помимо Promise активно **используются подходы на событиях и потоках-данных**
- Мы разберем эти подходы на отдельных лекциях


В АСИНХРОННОМ
ПРОГРАММИРОВАНИИ
НЕТ СМЫСЛА, ЕСЛИ
НЕТ ИНТЕНСИВНОГО
I/O?

В ТОЧКУ

- Асинхронное программирование не ускоряет код само по себе –
  напротив, **оно его даже зачастую замедляет**
- Если ваше **приложение не предполагает интерактивность** как таковую
  (запустил – получил результат), то в асинхронном программировании нет смысла
- Синхронный код проще и легче отлаживать,
  а **по общей производительности он может получиться быстрее**


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

В JS ОЧЕНЬ МНОГО АСИНХРОННОГО

- Потому что ЯП изначально создан для клиентских сценариев, где много различных **ожиданий действий пользователя и других асинхронных событий**
- Node.js – это серверная платформа, а для сервера важно оперативно **принимать новые запросы, не теряя интерактивности** (многозадачность)
- Но есть сценарии, где мы **спокойно можем обойтись и без него**

ОТКУДА БЕРУТСЯ ФУНКЦИИ ТИПА setTimeout?



ИХ ПРИНОСИТ СРЕДА

- **Виртуальная машина JS работает в паре со средой** – задача VM – исполнять JS-код, а задача среды – предоставлять функции для общения с внешним миром
- Если среда предоставляет асинхронный функционал, то она должна реализовывать **механизм регистрации обработчиков и очередь исполнения**
- **Браузер и Node.js** – это среды для JavaScript

У СРЕДЫ-БРАУЗЕРА ЕСТЬ СТРОГАЯ СПЕКА

- Она описывается «живым» стандартом HTML
- Именно в ней и описываются, в том числе, функции setTimeout
- Это крайне важно, так как сейчас существует множество браузеров-альтернатив, но **их поведение должно быть предсказуемым**
- **Node.js не подчиняется этому стандарту,**
  но многие его примитивы повторяют поведение браузера
- Это позволяет **предсказуемо запускать один и тот же код** в разных средах

АГЕНТЫ

- Каждая вкладка браузера или поток Node.js имеет **свой собственный глобальный объект, стек и поток исполнения**
- В спецификации такие сущности называются агентами
- Среда в таком случае **выступает в роли оркестратора** агентов

МОДЕЛЬ ПАМЯТИ

- Несмотря на возможность создания отдельных потоков в основных средах, **JS имеет однопоточный дизайн**
- В спецификации явно выделено, что каждый агент **исполняет свой код в одном потоке исполнения**
- Это важно, так как мы точно знаем, что ни один обработчик в нашем коде **не может работать параллельно с другим в рамках одного агента**
- Новые потоки (агенты) **создаются из отдельных файлов** с помощью API среды
- Например, `new Worker("file.js")`


РАЗВЕ
АСИНХРОННОЕ
НЕ ДОЛЖНО БЫТЬ
ПАРАЛЛЕЛЬНЫМ?

НЕТ

- Синхронный **по написанию** код может выполняться **параллельно** (многопоточно)
- А асинхронный код **может работать последовательно в одном потоке**
- **Возможен и обратный вариант** – поэтому никакой связи у асинхронного с многопоточностью нет
- Но **многие асинхронные среды используют потоки** для повышения производительности, например, Tokio в Rust

МЫ ЖЕ СПЕЦИАЛЬНО УХОДИЛИ ОТ ПОТОКОВ?



НЕ СОВСЕМ ТАК

- Создавать **много потоков, чтобы просто ждать** – это плохая стратегия
- Но **когда есть что исполнять**, то потоки будут очень кстати
- Например, если у сервера есть 16 ядер – значит, он может **параллельно обрабатывать 16 или больше запросов**
- При этом если обработка запроса требует сходить в БД или ещё куда, то **за счёт асинхронного подхода поток не будет заблокирован**
- Асинхронная среда может **равномерно распределять нагрузку** на все доступные ей ядра процессора

НО В JS НЕ ТАК

- **Спецификация запрещает** агенту запускать свой код, используя разные потоки исполнения
- Но допускается явно создать новый агент, если среда это поддерживает (Web Workers в браузере или worker_threads в Node.js)
- Однако сами операции (не JS код) **могут параллелиться автоматически**

```
// Запросы параллелятся, а обработчики — нет
fetch("https://foo.com/a").then(console.log);
fetch("https://foo.com/b").then(console.log);
```

НО МОЖНО РЕАЛИЗОВАТЬ САМОСТОЯТЕЛЬНО

- Написать **библиотеку с реализацией зелёных потоков**
- Которая будет **масштабировать их исполнение** на настоящие потоки
- Это позволит **повысить производительность** системы
- Как альтернативу потокам можно **просто создавать отдельные процессы**

```
function parallelSetTimeout(callback, delay, ...args) {
  // Генерируем уникальный ID для этого таймера
  const id = Date.now() + Math.random();

  // Создаем строку с кодом для воркера
  const workerCode = `
  const callback = ${callback.toString()};
  const args = ${JSON.stringify(args)};

  setTimeout(() => {
    const result = callback(...args);
    self.postMessage({ timerId: "${id}", result });
  }, ${delay});
 `;

  const blob = new Blob([workerCode], { type: 'application/javascript' });
  const workerUrl = URL.createObjectURL(blob);

  const worker = new Worker(workerUrl);
  worker.onmessage = cancel;

  return { id, worker, cancel };

  function cancel() {
    worker.terminate();
    URL.revokeObjectURL(workerUrl);
  }
}

// Множественные параллельные таймеры (порядок каждый раз разный)
parallelSetTimeout(() => console.log("Таймер 1"), 100);
parallelSetTimeout(() => console.log("Таймер 2"), 100);
parallelSetTimeout(() => console.log("Таймер 3"), 100);
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

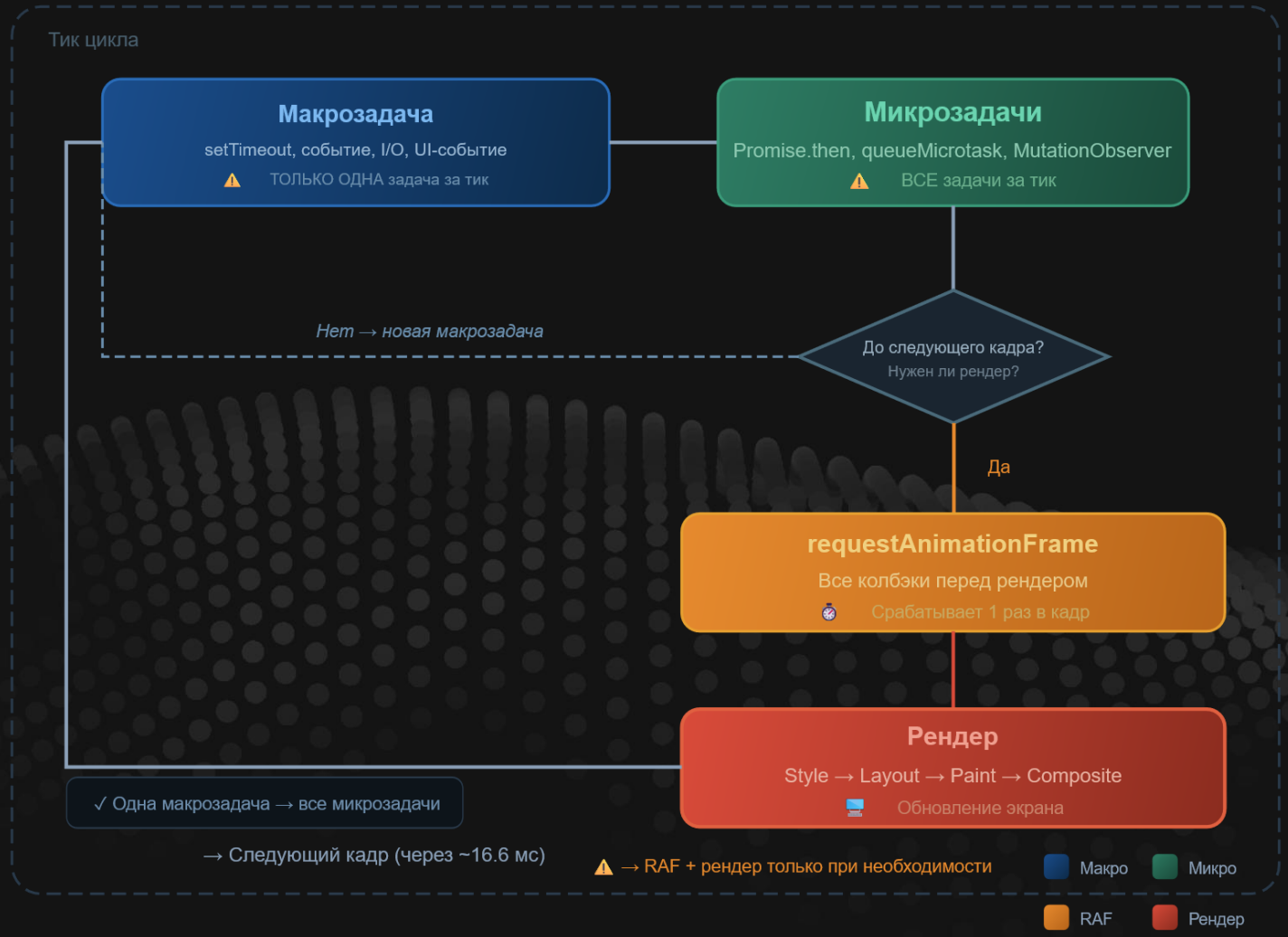
- Такие языки, как Go или Rust + Tokio, **могут быть куда эффективнее** и в асинхронной работе за счёт использования многопоточности
- Но такая производительность не всегда нужна, а **сложность разработки и отладки возрастает**
- **В JS есть настоящие потоки без GIL** (Global Interpreter Lock), но они создаются явно – это компромиссное решение
- Мы можем **разработать свою реализацию** зелёных потоков

РАБОТУ АГЕНТА В БРАУЗЕРЕ

- **Можно представить в виде цикла:** выполняются задачи, стоящие в очереди, ставятся новые, и при необходимости всё может заканчиваться отрисовкой (рендерингом)
- Затем **цикл повторяется снова и снова** – каждый его этап строго следует порядку, описанному в спецификации HTML Living Standard
- В идеале отрисовка должна выполняться столько раз в секунду, чтобы **соответствовать частоте обновления экрана**
- Данный цикл **именуется Event Loop браузера**
- Каждый агент **имеет свой Event Loop**

Цикл событий браузера (Event Loop)

Один кадр анимации (~16.6 мс)



ВСЕ МИКРОЗАДАЧИ ПОСЛЕ МАКРОЗАДАЧИ

```
setTimeout(() => {
  console.log(1);

  queueMicrotask(() => {
    console.log(3);

    queueMicrotask(() => {
      console.log(4);
    });
  });
}, 0);

setTimeout(() => {
  console.log(2);
}, 0);
```

ВСЕ МИКРОЗАДАЧИ ПОСЛЕ МАКРОЗАДАЧИ

```
setTimeout(() => {
  console.log(1);

  queueMicrotask(() => {
    console.log(3);

    queueMicrotask(() => {
      console.log(4);
    });
  });
}, 0);

setTimeout(() => {
  console.log(2);
}, 0);
```

ВСЕ МИКРОЗАДАЧИ ПОСЛЕ МАКРОЗАДАЧИ

```
setTimeout(() => {
   console.log(1);

   queueMicrotask(() => {
      console.log(3);

      queueMicrotask(() => {
         console.log(4);
      });
   });
}, 0);

setTimeout(() => {
   console.log(2);
}, 0);
```

ВСЕ МИКРОЗАДАЧИ ПОСЛЕ МАКРОЗАДАЧИ

```
setTimeout(() => {
  console.log(1);

  queueMicrotask(() => {
    console.log(3);

    queueMicrotask(() => {
      console.log(4);
    });
  });
}, 0);

setTimeout(() => {
  console.log(2);
}, 0);
```

ВСЕ МИКРОЗАДАЧИ ПОСЛЕ МАКРОЗАДАЧИ

```
setTimeout(() => {
   console.log(1);

   queueMicrotask(() => {
      console.log(3);

      queueMicrotask(() => {
         console.log(4);
      });
   });
}, 0);

setTimeout(() => {
   console.log(2);
}, 0);
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ЧТО ТУТ НАИБОЛЕЕ ВАЖНО

- Никакой параллельности в рамках агента –
  любой долго выполняемый обработчик тормозит весь цикл
- Микрозадачи/микротаски – это **асинхронные задачи с наивысшим приоритетом**
  (в браузере; в Node.js есть свои нюансы)
- Очередь микрозадач опустошается **перед выполнением следующей макрозадачи**
- За **один тик – исполняется одна макрозадача**
- Загрузка и исполнение <script> – макрозадача

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

```
console.log(1);

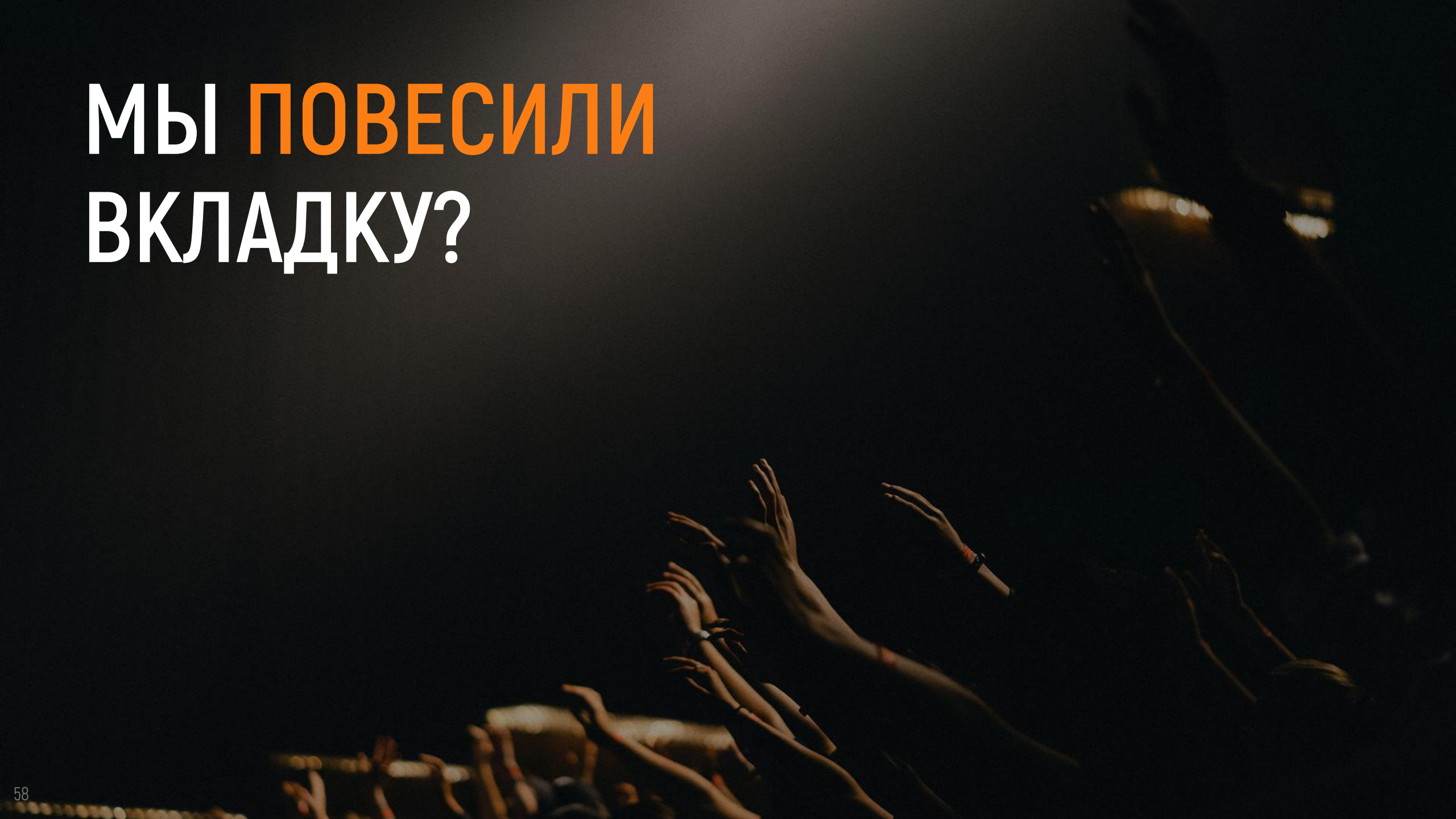
setTimeout(() => {
    console.log(2);
}, 0);

function pushMicrotask() {
    queueMicrotask(() => {
        console.log(3);
        pushMicrotask(); // Бесконечно добавляем микрозадачи в очередь
    });
}

pushMicrotask();

console.log(4);
```

МЫ ПОВЕСИЛИ ВКЛАДКУ?



В ТОЧКУ

- Добавляя новую микрозадачу из другой микрозадачи,
  мы **добавляем её в конец текущей очереди**
- Это можно описать как **«выполни этот код, после того как выполнишь то, что синхронно исполняется сейчас»**
- **Пока все текущие микрозадачи не выполнятся**, никакие макрозадачи (обработчики событий, рендеринг) выполняться не будут
- Если вы хотите **отложить задачу «на потом» и не блокировать поток** — используйте макрозадачи

А ЗАЧЕМ ОНО ВООБЩЕ НУЖНО?



РАЗДЕЛЕНИЕ ПОРЯДКА ИСПОЛНЕНИЯ

- У нас **есть синхронный порядок исполнения кода**
- Мы можем добавить обработчик, **который выполнится после синхронного**
- Но **ДО любой макрозадачи** (обработка действий пользователя, таймеры, рендеринг)
- Используя этот подход можно **создавать удобные и эффективные API**

```
// Функция, которую слишком часто вызывать не хочется
function rerenderApp() {
  // Очень нагружающая браузер задача, например, DOM
  heavyTask();
}

function deferred(fn) {
  let planned = false;

  return () => {
    if (planned) {
      return;
    }

    planned = true;
    queueMicrotask(() => {
      planned = false;
      fn();
    });
  };
}
```

```
const deferredRendering = deferred(rerenderApp);

function doSomething1() {
  // ...
  // Спокойно вызываем, так как смежные вызовы схлопнутся
  deferredRendering();
}

function doSomething2() {
  // ...
  // Спокойно вызываем, так как смежные вызовы схлопнутся
  deferredRendering();
}

doSomething1();
doSomething2();
```

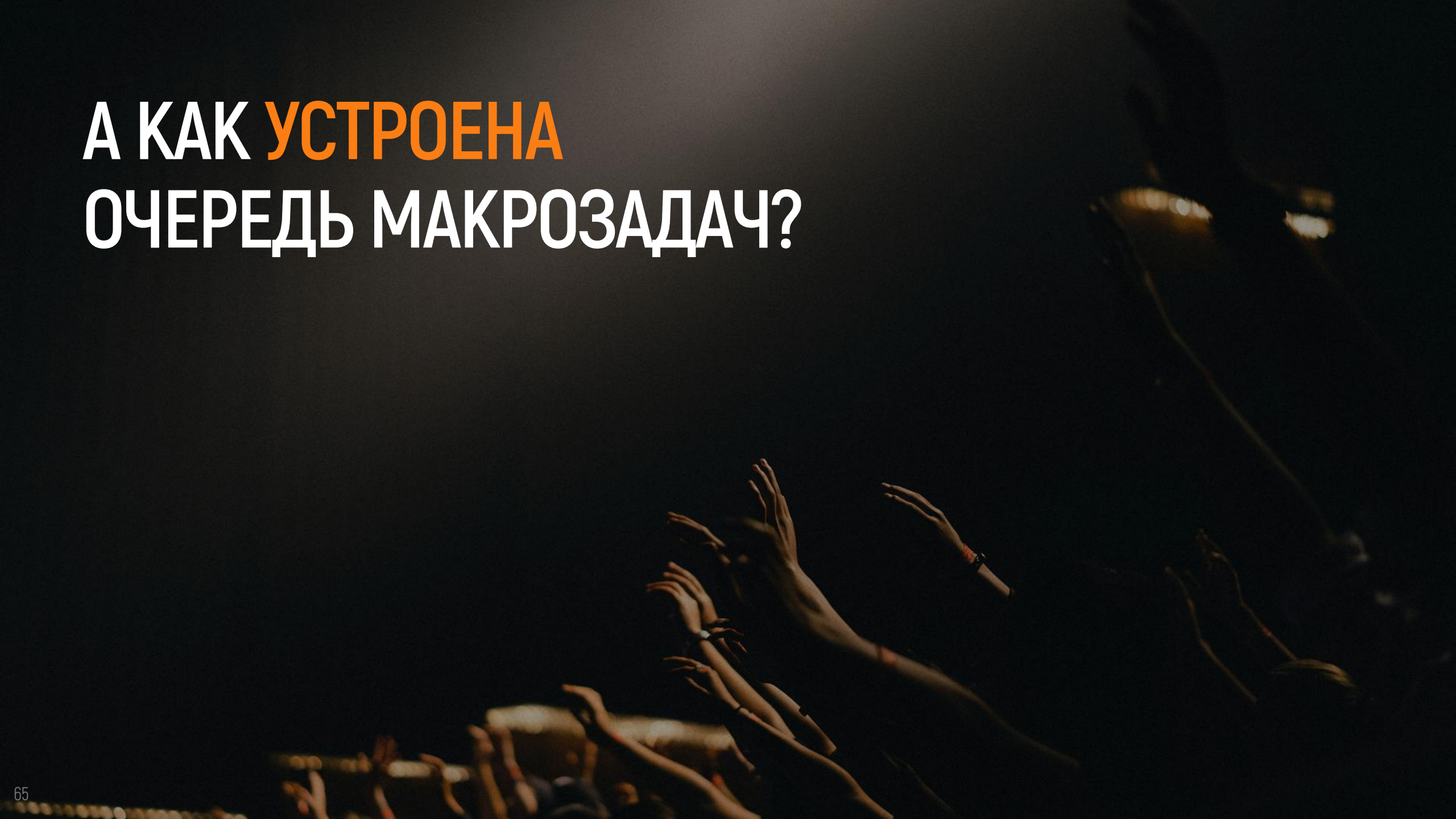
РЕФЛЕКСИРУЕМ

- Мы схлопнули смежные вызовы функции в один –
  тем самым **избежали лишней работы** (и улучшили производительность)
- Например, MutationObserver не вызывает обработчик на каждое изменение DOM,
  а **схлопывает все изменения в один вызов за тик Event Loop**
- Обработчик ставится в очередь микрозадач

МИКРОЗАДАЧИ ИСПОЛЬЗУЮТ

- Все методы объекта Promise – даже если «данные уже на руках», **обработчики всё равно выполнятся асинхронно**
- MutationObserver тоже использует микротаски
- Можно явно добавить функцию в очередь микрозадач через queueMicrotask
- **Многие популярные библиотеки** активно используют этот подход

А КАК УСТРОЕНА ОЧЕРЕДЬ МАКРОЗАДАЧ?



ОЧЕРЕДЕЙ МНОГО

- И **у каждой есть свой приоритет** — одни важнее других
- В спецификации **вместо термина «макрозадача»** используется «задача» и «очередь задач»
- Более того, появился API планировщика с возможностью **добавления задач в конкретную очередь**

Приоритеты очередей задач в браузере

Порядок обработки в цикле событий (от высшего к низшему)

Уровень 1 — Высший

Микрозадачи

Promise.then · queueMicrotask · MutationObserver

✓ *Выполняются после КАЖДОЙ макрозадачи, ДО рендера*

Уровень 2 — Высокий

user-blocking (explicit)

Критичный рендеринг · Обработка ввода пользователя

⚠ *Важнее обычных user-visible, но ниже микрозадач*

Уровень 3 — Средний

user-visible (по умолчанию) + Интерактивные события

Обработчики click/keydown/scroll · user-visible задачи

✓ *Обеспечивает отзывчивость интерфейса после критичных операций*

Уровень 4 — Низкий

Таймеры + background задачи

setTimeout/setInterval (обычные) · background задачи (логи, аналитика)

⚠ *setTimeout(fn, 0) встаёт в конец этой очереди*

Уровень 5 — Низший

requestIdleCallback

Фоновые задачи · Некритичные операции

✓ *Выполняется только когда главный поток полностью свободен*

Приоритет →

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Критически важная задача");
    return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
    console.log("Фоновая задача");
    return 0;
}, { priority: "background" }).then(console.log, console.error);
```

API ПЛАНИРОВЩИКА

```
scheduler.postTask(() => {
   console.log("Обычная задача");
   return 100500;
}).then(console.log, console.error);

scheduler.postTask(() => {
   console.log("Критически важная задача");
   return 42;
}, { priority: "user-blocking" }).then(console.log, console.error);

scheduler.postTask(() => {
   console.log("Фоновая задача");
   return 0;
}, { priority: "background" }).then(console.log, console.error);
```

ДИНАМИЧЕСКАЯ ПРИОРИТЕЗАЦИЯ

```
scheduler.postTask(() => {
   console.log("Обычная задача");
   return 100500;
}).then(console.log, console.error);

const controller = new TaskController();

scheduler.postTask(
   () => console.log("Внезапная задача"),
   { signal: controller.signal }
);

// Меняем приоритет при необходимости
controller.setPriority("user-blocking");
```

ДИНАМИЧЕСКАЯ ПРИОРИТЕЗАЦИЯ

```
scheduler.postTask(() => {
   console.log("Обычная задача");
   return 100500;
}).then(console.log, console.error);

const controller = new TaskController();

scheduler.postTask(
   () => console.log("Внезапная задача"),
   { signal: controller.signal }
);

// Меняем приоритет при необходимости
controller.setPriority("user-blocking");
```

ДИНАМИЧЕСКАЯ ПРИОРИТЕЗАЦИЯ

```
scheduler.postTask(() => {
    console.log("Обычная задача");
    return 100500;
}).then(console.log, console.error);

const controller = new TaskController();

scheduler.postTask(
    () => console.log("Внезапная задача"),
    { signal: controller.signal }
);

// Меняем приоритет при необходимости
controller.setPriority("user-blocking");
```

РАЗБИЕНИЕ НА ПОДЗАДАЧИ

```
scheduler.postTask(async () => {
  console.log(1);
  await scheduler.yield(); // Продолжение унаследует "background"
  console.log(2);
}, { priority: "background" });
```

ОТМЕНА ЗАДАЧИ

```
const controller = new TaskController();

scheduler.postTask(
    () => console.log("Внезапная задача"),
    { signal: controller.signal }
).catch(console.error);

// Отменяем задачу
controller.abort();
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Event Loop браузера проделал большой путь от отдельных API типа setTimeout до **явного API планировщика задач** (см. поддержку в браузерах)
- Новый API позволяет **точечно ставить задачу с конкретным приоритетом** и даже менять его на лету или отменять
- И предоставляет **примитивы для организации кооперативной многозадачности**
- Это очень важно **для создания отзывчивых интерфейсов**

А КАК ДЕЛА В NODE.JS?



ТОЖЕ EVENT LOOP, НО СВОЙ

- Node.js позволяет прямую **работу с файловой системой и другими примитивами ОС**
- С другой стороны, нет рендеринга или других браузерно-специфичных вещей
- Ещё **нет аналога API планировщика**
- Но подход с Event Loop и **микрозадачами** очень похож
- Технически **реализуется с помощью библиотеки** libuv
  (микрозадачи — на стороне V8)

```
queueMicrotask(() => {
  console.log(1);

  queueMicrotask(() => {
    console.log(2);
  });
});

process.nextTick(() => {
  console.log(3);

  process.nextTick(() => {
    console.log(4);
  });
});
```

```
queueMicrotask(() => {
    console.log(1);

    queueMicrotask(() => {
        console.log(2);
    });
});

process.nextTick(() => {
    console.log(3);

    process.nextTick(() => {
        console.log(4);
    });
});
```

```
queueMicrotask(() => {
    console.log(1);

    queueMicrotask(() => {
        console.log(2);
    });
});

process.nextTick(() => {
    console.log(3);

    process.nextTick(() => {
        console.log(4);
    });
});
```

```
queueMicrotask(() => {
  console.log(1);

  queueMicrotask(() => {
    console.log(2);
  });
});

process.nextTick(() => {
  console.log(3);

  process.nextTick(() => {
    console.log(4);
  });
});
```

```
queueMicrotask(() => {
    console.log(1);

    queueMicrotask(() => {
        console.log(2);
    });
});

process.nextTick(() => {
    console.log(3);

    process.nextTick(() => {
        console.log(4);
    });
});
```

process.nextTick VS queueMicrotask

- Обе откладывают колбэк до конца текущей операции,
  но **живут они в разных очередях**
- Очередь nextTick имеет **более высокий приоритет**
- Но **есть нюанс** с файлами ESM-модулей*
- **Предпочитайте обычные микротаски**, так как в браузере нет *process*.nextTick

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

```
setTimeout(() => console.log(1), 0);
setTimeout(() => console.log(2), 0);

setImmediate(() => console.log(3));
setImmediate(() => console.log(4));

setTimeout(() => console.log(5), 0);
```

setImmediate VS setTimeout(0)

- Обе эти функции создают макрозадачи, но **живут они в разных очередях**
- Колбэк setImmediate срабатывает раньше **setTimeout(0) только внутри I/O-цикла – вне его порядок не гарантирован**
- **В браузере его также нет** – используйте, только когда знаете зачем

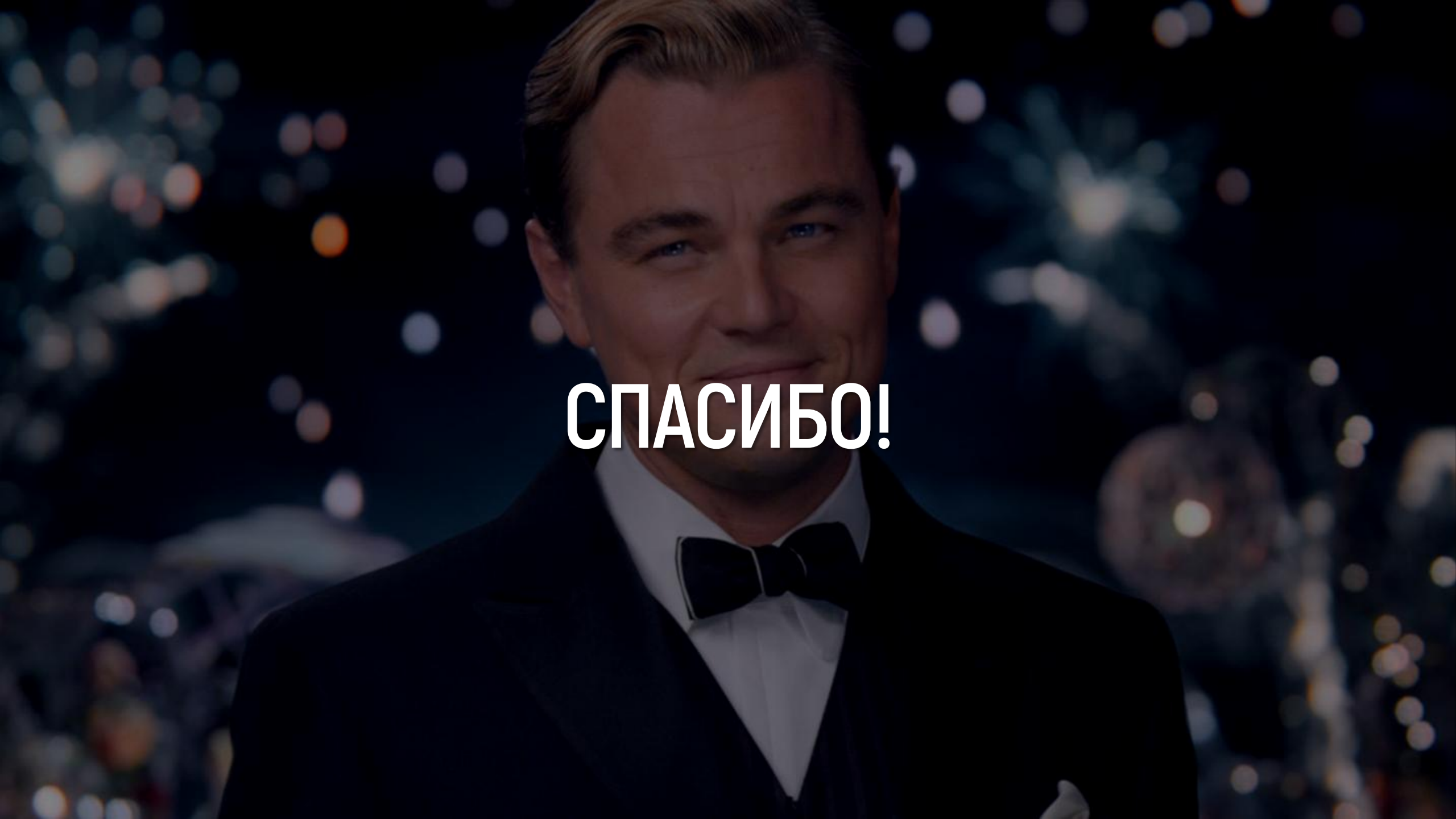

second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- **Node.js появился раньше**, чем в браузерах стандартизировали Event Loop и появились нативные промисы и микротаски
- Поэтому есть **некоторый слой легаси**
- Современный тренд Node.js – **повторять насколько это возможно** поведение кода в браузере
- При этом держим в голову, что **в Node.js свой Event Loop** со своей спецификой (файлы, сеть и другое)

ПОДВЕДЕМ ИТОГИ

- Асинхронное программирование необходимо для более эффективной работы **в сценариях простоя из-за ожидания ввода-вывода** (I/O)
- Минусом является запутанный порядок исполнения: код, описанный ниже, **может выполниться раньше или не выполниться вовсе**
- Для работы асинхронного кода **необходима специальная среда**
- JavaScript, будучи встраиваемым языком, описывает в спецификации модель памяти и то, как среда может взаимодействовать с VM
- В браузере есть строгий стандарт (HTML Living Standard), который определяет Event Loop, очереди задач и микрозадач


СПАСИБО!